

情報科学演習D

課題3

担当教員 : 梶井 晃基
提出者 : 鶴見 俊介
所属/学年 : 基礎工学部 情報科学科 3年 計算機科学コース
学籍番号 : 09B23057
電子メール : u299151f@ecs.osaka-u.ac.jp

提出日 : 2025 年 12 月 12 日
締切日 : 2025 年 12 月 12 日

1 システムの仕様

本システムは、Pascal 風言語プログラムの字句解析結果（ts ファイル）に対して構文解析および意味解析を行い、結果を文字列で返す意味解析器である。エントリポイントは `enshud.s3.checker.Checker` クラスの `run(String)` メソッドとする。

`run` メソッドは、入力として ts ファイル名 `inputFileName` を受け取り、次のいずれかの文字列を返す。

- 入力ファイルが存在しない場合："File not found"
- 構文解析で最初に検出された構文誤りの行が `n` のとき："Syntax error: line `n`"
- 構文的に正しい場合に、意味解析で最初に検出された意味誤りの行が `n` のとき："Semantic error: line `n`"
- 構文・意味のいずれの解析でも誤りが検出されなかった場合："OK"

2 課題達成の方針と設計

2.1 設計方針とクラス構成

本課題の意味解析器は、構文解析で得られた AST を入力として、Visitor パターンにより 1 回の走査で宣言チェックと型検査を行う方針とした。Checker.run はまず ts ファイルを読み込み、構文解析器を用いて Program ノードを根とする AST を生成する。この段階で構文誤りが検出された場合には意味解析を行わず、メッセージ "Syntax error: line `n`" を返す。構文的に正しいければ Program ノードを意味解析器 Semantic に渡し、意味解析を開始する。

図 1 に、意味解析に関わる主要クラスの関係を示す。Semantic は AstVisitor インタフェースを実装していて、Program.accept(this) を起点として各種 visit* メソッドを通じて AST 全体を巡回する。識別子情報は SymbolTable が管理し、Symbol には name, kind, type, paramTypes などを持させることで、変数・仮引数・手続きの宣言と参照を一元的に扱えるようにした。また、Type クラスは組み込み型や配列型を表現し、各式ノードに対する visit* の戻り値として使用することで、代入文、条件式、演算子、手続き呼び出しなどの型整合性検査を一貫して行う。

図 2 は、Checker と Semantic、SymbolTable を中心とした意味解析処理の流れをシーケンス図として表したものである。Checker が Semantic を生成すると、Semantic は最初に SymbolTable.enterScope() でグローバルスコープを作成し、続いて Program.accept(this) により AST 走査を開始する。各 visit* メソッド内では、宣言ノードに対して識別子の登録と重複宣言の検査を行い、式ノードに対しては Type を計算して親ノードに返すことで、上位構造の型チェックに利用する。いずれかの visit* で意味的な誤りを検出した場合には CheckError 例外を送出し、Checker.run がこれを捕捉して最初の 1 件のみを "Semantic error: line `n`" として報告する。

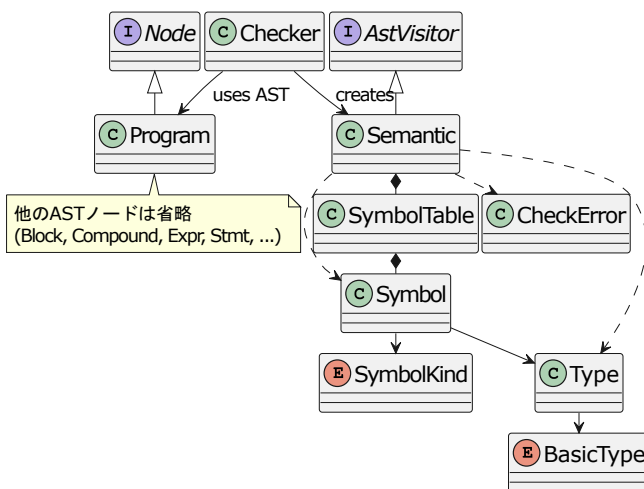


図 1: 意味解析に関わるクラス図

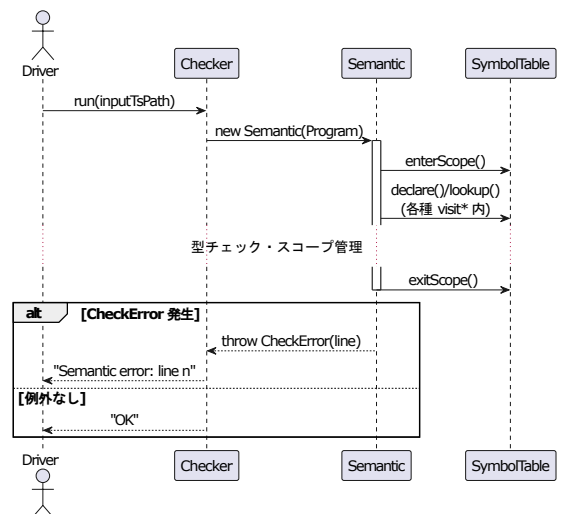


図 2: 意味解析処理のシーケンス図

2.2 スコープ管理とシンボル表の設計

本実装では、プログラム本体および手続き宣言の入れ子構造に対応して、図3(a)のような静的スコープ構造を前提とする。各スコープは、その領域で有効な識別子(変数・仮引数・手続き名)と対応する **Symbol** 情報 (**name**, **kind**, **type** など) を保持する単位であり、グローバルスコープの内部に手続き A のスコープ、さらにその内部に手続き B のスコープが入れ子になる形で表現される。

SymbolTable は、この静的スコープ構造を図3(b)のような「スコープのスタック」として管理する。**Semantic** が **AST** を走査する際に、プログラム開始時および各手続きの本体に入る直前で **enterScope()** を呼び出し、対応するスコープフレームをスタックに **push** する。手続き本体の末尾やプログラム末尾では **exitScope()** によりフレームを **pop** し、スコープから抜けた識別子は以降の名前解決の対象外となる。

宣言ノードに対しては、現在スコープに対して **declare()** を呼び出し、同一スコープ内に同名シンボルが存在する場合は重複宣言として **CheckError** を送出する。一方、変数・配列・手続き呼び出しなど参照側のノードでは **lookup()** を用いて、スタックトップから順に外側スコープへ探索することで静的スコープ規則に従った名前解決を実現し、見つからなかった場合は未宣言識別子のエラーとして扱う。

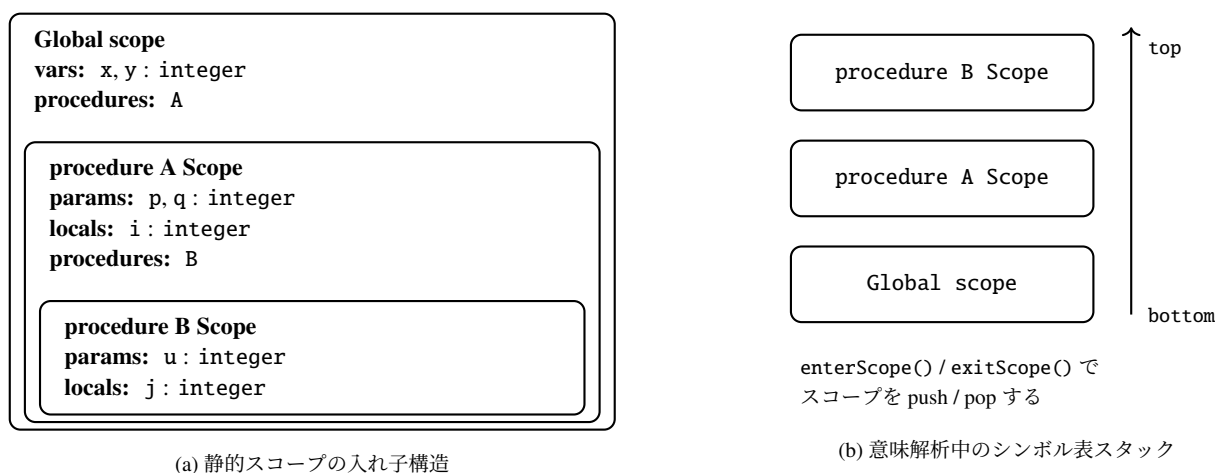


図 3: 静的スコープ構造と意味解析時のシンボル表スタックの対応

2.3 検出対象エラー

表 1: 検出対象エラーと主な検出箇所

エラー種別	検出方針
変数・仮引数・手続き名の重複宣言	SymbolTable.declare で同一スコープ内の重複を検査し、重複時に CheckError を送出する。
未宣言の変数・手続き名の参照	各種 visit* から SymbolTable.lookup を呼び、未登録であればエラーとする。
配列以外に対する添字アクセス／添字式が整数型でない	visitArrayAccess , visitArrayRef で参照先の Type が配列型かどうかと、添字式の型が integer かどうかを検査する。
代入文の左辺が配列型／左右の型不一致	visitAssign で左辺の isArray と左右の Type を確認し、不適切な場合にエラーとする。
if / while 条件式が boolean 以外	visitIfS , visitWhileS で条件式の型が boolean でない場合にエラーとする。
演算子と被演算子の型不整合	visitBinary , visitUnary で演算子ごとの許容型をチェックし、違反時にエラーとする。
手続き呼び出しの引数個数・型の不一致	visitProcCall で Symbol.paramTypes と実引数列を比較し、個数・型のいずれかが不一致であればエラーとする。

本システムでは、課題文で要求されている代表的な意味的誤りをすべて検出対象とした。そのうえで、各エラー種別について「どの `visit*` / `SymbolTable` メソッドで検査するか」をあらかじめ整理し、実装時に抜け漏れが生じないようにした。表 1 に、エラー種別と主な検出箇所の対応を示す。

3 実装プログラム

3.1 Visitor パターンによる意味解析クラス

本実装では、意味解析器 `Semantic` を AST 全体に適用するために、`Visitor` パターンを用いた。`Semantic` のコンストラクタ（プログラム 1）では、まずグローバルスコープを作成した上で、`Program` ノードに対して `prog.accept(this)` を呼び出し、意味解析を開始する。

プログラム 1: `Semantic` コンストラクタ

```
1 public Semantic(Program prog) {
2     symtab.enterScope();
3     prog.accept(this);
4     symtab.exitScope();
5 }
```

`Program` クラス側の `accept`（プログラム 2）は、渡された `AstVisitor` に対して `visitProgram(this)` を呼ぶだけの薄いラップであり、「どの `visit*` を呼ぶか」をノード側が決定する。`Semantic` の `visitProgram`（プログラム 3）では、`block` と `main` に対して再び `accept(this)` を呼び出し、下位のノードへ処理を委譲する。この再帰を通じて、AST 全体が 1 回走査される。

プログラム 2: `Program` クラス

```
1 public record Program(String name, Block block, Compound main, int line) implements Node {
2     public <R> R accept(AstVisitor<R> v) return v.visitProgram(this);
3 }
```

プログラム 3: `visitProgram` メソッド

```
1 public Type visitProgram(Program node) {
2     if (node.block() != null) node.block().accept(this);
3     if (node.main() != null) node.main().accept(this);
4     return null;
5 }
```

`Program` ノード自身は型を持たないため `visitProgram` の戻り値は `null` としているが、式ノードの `visit*` は `Type` を返し、文ノードの `visit*` は `null` を返す、という方針で統一している。このように、「`Checker` → `Program.accept(this)` → 各 `visit*` → 子ノードの `accept(this)` → ...」という流れを取ることで、AST の走査手順は `accept` / `visit*` の呼び分けに集約され、意味解析の個々の処理（シンボル表操作や型検査）は各 `visit*` メソッドの中に局所化されるようになっている。また、新たな解析処理を追加する場合でも、既存の AST クラスを変更せずに済むといった利点が得られる。

3.2 スコープ付きシンボル表の実装

識別子の宣言と参照を扱うために、`Symbol`、`Type`、`SymbolTable` を用意した。`Type` は基底型 (`BasicType`) と配列フラグ・添字範囲をまとめた型情報、`Symbol` は識別子名 (`name`)、種別 (`kind`: 変数 / 仮引数 / 手続き)、型 (`type`)、宣言位置 (`line`) などを保持する。`SymbolTable` は「`Map<String, Symbol>` のスタック」として実装し、`enterScope()` / `exitScope()` でスコープ境界を `push` / `pop` することで、図 3 のような静的スコープ構造を表現している。

宣言時には現在スコープに対して `declare(Symbol sym)` を呼び、同一スコープ内に同名シンボルがあれば `CheckError` を送出する。参照時には `lookup(String name, int line)` によりスタックのトップから順に探索し、最初に見つかった `Symbol` を返すことで、外側のスコープも含めた名前解決と未宣言識別子の検出を行う。

意味解析側では、各種 `visit*` からこのシンボル表を直接呼び出す。例えば変数宣言ノード `VarDecl` に対する `visitVarDecl` (プログラム 4) では、まず `resolveType` で宣言型を `Type` に変換し、`x, y, z: integer;` のような複数同時宣言に対して、各名前ごとに `Symbol` を生成して `syntab.declare` で登録する。手続き宣言や仮引数宣言も同様に `declare` で登録し、代入文や式ノード側では `lookup` を通じて `Symbol` を取得して型検査に利用する、という役割分担になっている。

プログラム 4: `VarDecl` の意味解析

```
1 public Type visitVarDecl(VarDecl node) {
2     Type t = resolveType(node.type(), node.line());
3     for (String name : node.names()) {
4         Symbol sym = new Symbol();
5         sym.name = name;
6         sym.kind = SymbolKind.VAR;
7         sym.type = t;
8         sym.line = node.line();
9         syntab.declare(sym);
10    }
11    return null;
12 }
```

4 考察や工夫点

本課題では、Parser による AST 構築を前提に、意味解析を `Checker`・`Semantic`・`SymbolTable` の 3 クラスに分離した。`Checker` は入出力と例外処理のみを担当し、実際の走査と検査は `Semantic`、識別子管理は `SymbolTable` に委譲することで、構文解析と意味解析の責務分離と、意味規則の追加・変更の局所化を図った。

スコープ管理については、図 3 のような静的スコープ構造を、そのまま `SymbolTable` 内のスコープスタックに写像する設計とした。`enterScope()` / `exitScope()` によりスコープの `push` / `pop` を行い、`declare()` は「現在スコープ内での重複宣言のみ」を検出する。一方 `lookup()` はスタックのトップからボトムへ線形探索することで、内側の宣言が外側を隠す静的スコープ規則を自然に実現している。これにより、入れ子の手続き宣言に対しても、単純なスタック操作で一貫した名前解決が行えるようになった。

型検査では、配列型を含む全ての型情報を `Type` に集約し、基底型 `base` (`integer` / `char` / `boolean`) と配列フラグ `isArray`、添字範囲 `lower`, `upper` を持たせた。特に、`visitArrayAccess` で配列変数に対する添字アクセスの際に要素型 (`elementType()`) を返す設計としたことで、`visitAssign` では「`a[i]` の型」と右辺式の型を単純に比較するだけで型一致判定ができるようになっている。各 `visit*` が返す `Type` の意味を慎重に決めたことが、全体の実装を簡潔に保つうえで重要であった。

手続き呼び出しの検査では、`ProcedureDecl` の仮引数グループ (例: `(Y, M, D: integer)`) を展開し、各名前ごとに同じ `Type` を `Symbol.paramTypes` に順序付きで登録するようにした。これにより、`visitProcCall` では `Symbol.paramTypes` と実引数リストを 1 対 1 で対応付けて個数と型の両方を比較できる。途中で仮引数グループ単位で型リストを保持してしまい、引数比較に失敗するバグも生じたが、`Param` の構造と `Symbol` の保持形式を見直すことで整合した設計に修正できた。

5 感想

本課題を通じて、教科書で見ていた静的スコープやシンボル表、`Visitor` パターンによる AST 走査といった概念を、自分で実装して動かせるレベルまで具体化できたと感じた。特に、`Type` や `Param` の設計では、最初に安易な表現を選ぶと、後段の型比較や引数チェックが急速に複雑化して破綻することを経験し、抽象データ構造の設計がコンパイラ実装の成否を大きく左右することを実感した。

また、構文解析で構築した AST を意味解析のみにとどまらず、拡張版との共通基盤として使い回せる形で整理できたのは大きな収穫である。一方で、今回は変数の初期化状態や配列添字の範囲といった実行時情報に依存する誤りは扱っておらず、データフロー解析などを導入することで、より高度な意味解析へ拡張できる余地も見えた。今後は、今回の設計を土台として、より高機能な検査やコンパイラ全体の構築にも発展させていきたい。